Laporan Milestone 1

IF4031 Arsitektur Aplikasi Terdistribusi

Disusun oleh Kelompok SixtributedSystems

Rasyad Satyatma	13524142
Bernhard Aprillio Pramana	13524074
Kenneth Ricardo Chandra	13523022

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

DAFTAR ISI

DAFTAR ISI 1
Tabel kontribusi 1
Deskripsi umum sistem 2
2.1. Latar Belakang dan Tujuan 2
2.2. Lingkup Milestone 1 2
2.3. Komponen Sistem 3
Diagram Arsitektur Sistem 4
3.1. Diagram 4
3.2 Narasi Arsitektur 4
3.2.1. Seluruh Service dan Batas Container 5
3.2.2. Alur Event (Data Flow) 5
3.2.3. Kepemilikan Penyimpanan (Storage Ownership) 5
3.2.4. Protokol Komunikasi Antar Service 5
Daftar Teknologi Terpilih 7
Asumsi 9
Petunjuk Menjalankan Sistem 10
6.1. Prasyarat 10
6.2. Menyalakan Sistem 10
6.3. Health Check 10
6.4. Menjalankan Skenario Demonstrasi 10
Problem 1: Arsitektur & Interoperabilitas Data 12
7.1. Rancangan Solusi dan Mekanisme yang Dipilih 12
7.2. Alasan Keputusan dan Trade-off 12
7.3. Batasan yang Disadari 12
7.4. Bukti Penyelesaian 13
7.5. Jawaban atas Butir Di Laporan 14
Problem 2: Concurrency, Availability, & Graceful Degradation 15
8.1. Rancangan Solusi dan Mekanisme yang Dipilih 15
8.2. Alasan Keputusan dan Trade-off 15
8.3. Batasan yang Disadari 15
8.4. Bukti Penyelesaian 15
8.5. Jawaban atas Butir Di Laporan 17
Problem 3: Service-to-Service Authentication & Trust 18
9.1. Rancangan Solusi dan Mekanisme yang Dipilih 18
9.2. Alasan Keputusan dan Trade-off 18
9.3. Batasan yang Disadari 18
9.4. Bukti Penyelesaian 18
9.5. Jawaban atas Butir Di Laporan 21
Problem 4: Arsitektur Microservice & Flexible Storage 22

10.1. Rancangan Solusi dan Mekanisme yang Dipilih..... 22
10.2. Alasan Keputusan dan Trade-off..... 22
10.3. Batasan yang Disadari..... 22
10.4. Bukti Penyelesaian..... 22
10.5. Jawaban atas Butir Di Laporan..... 24
Problem 5: Fan-out Event Distribution / Pub-sub..... 25
11.1. Rancangan Solusi dan Mekanisme yang Dipilih..... 25
11.2. Alasan Keputusan dan Trade-off..... 25
11.3. Batasan yang Disadari..... 25
11.4. Bukti Penyelesaian..... 25
11.5. Jawaban atas Butir Di Laporan..... 28
DAFTAR PUSTAKA..... 29

Tabel kontribusi

Nama	NIM	Kontribusi
Rasyad Satyatma	13524142	Implementasi sistem, testing
Bernhard Aprillio Pramana	13524074	Pengerjaan laporan, testing
Kenneth Ricardo Chandra	13523022	

Deskripsi umum sistem

2.1. Latar Belakang dan Tujuan

Sistem ini merupakan purwarupa *gateway* dan platform koordinasi kebencanaan terdistribusi yang dibangun untuk BNPB. Tujuan utama sistem ini adalah menjembatani dan mengintegrasikan dua sumber data kebencanaan yang terpisah dan independen, yaitu BMKG (untuk pemantauan gempa dan potensi tsunami) serta PVMBG (untuk pemantauan aktivitas gunung api).

Karena masing-masing instansi sumber memiliki infrastruktur, format data, dan keandalan yang berbeda (seperti infrastruktur PVMBG yang disimulasikan lebih lawas dan fluktuatif), sistem ini didesain agar *fault-tolerant*, terhindar dari *head-of-line blocking*, serta mampu menstandarisasi data heterogen tersebut ke dalam satu bentuk format data kanonik (*HazardEvent*). Platform ini juga bertujuan mendistribusikan data secara aman dengan batasan hak akses kepada berbagai klien *downstream* (Media, Tim Lapangan, dan Internal Pusat) serta menyebarkan notifikasi secara *real-time* kepada layanan konsumen melalui arsitektur *Pub/Sub*.

2.2. Lingkup Milestone 1

Lingkup yang dicakup:

- Dua *mock service* sumber (BMKG dan PVMBG) yang bersifat pasif dan hanya merespons permintaan REST API.
- Dua jenis bencana spesifik: seismik (BMKG) dan vulkanik (PVMBG).
- Pola penarikan data (*ingest*) menggunakan *polling* berkala dari komponen Aggregator BNPB menuju kedua *mock service*.
- Mekanisme pemetaan dinamis (skema aditif) untuk menerjemahkan data mentah ke entitas terpadu *HazardEvent*.
- Implementasi *Canonical Store* yang eksklusif dikelola oleh *Aggregator*.
- Layanan API klien (*Client-Facing API*) yang diotorisasi menggunakan *access token* ber-TTL pendek (60 detik) dengan pemisahan *domain of trust*.
- Distribusi *event* kanonik ke minimal dua komponen *consumer* independen menggunakan instrumen *message broker*.
- Orkestrasi kontainerisasi menggunakan Docker dan dikelola melalui konfigurasi tunggal docker-compose.yml.

Di luar lingkup:

- **Pola Push Event dari Sumber:** Sistem sumber (BMKG/PVMBG) tidak melakukan pengiriman *event* otomatis (push/webhook) ke BNPB; aliran data sepenuhnya bertumpu pada inisiatif *polling* dari sisi *gateway*.
- **Jalur Pelaporan Lapangan:** Sistem klien bersifat *read-only*. Alur pengumpulan atau penulisan data langsung dari unit lapangan (contoh: laporan manual warga) tidak termasuk dalam implementasi M1.

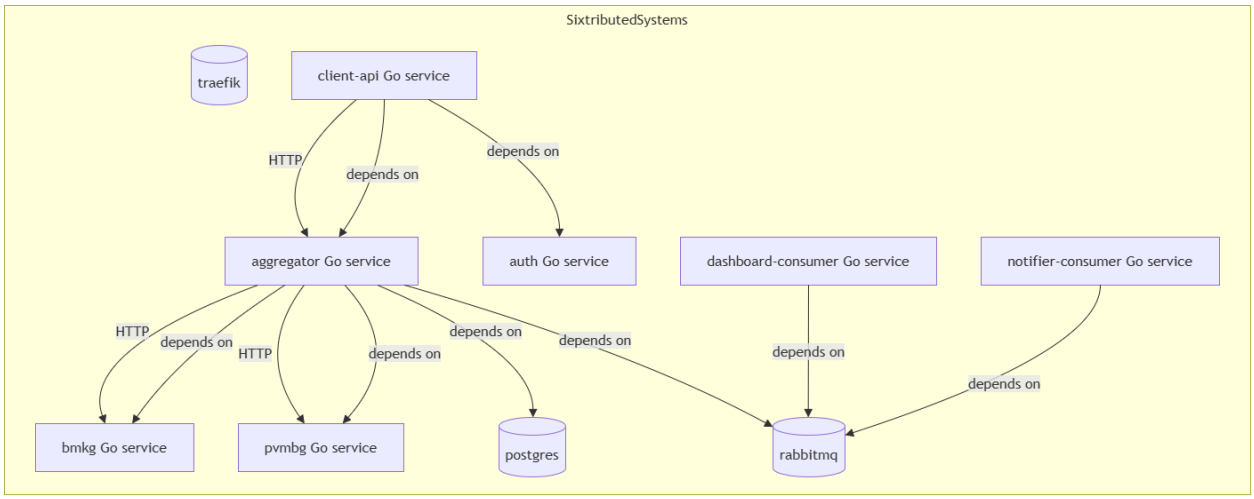
- Klasterisasi Geografis: Seluruh komponen disimulasikan berjalan pada satu titik mesin orkestrasi (melalui Docker Compose lokal) tanpa simulasi latensi jaringan fisik eksternal, selain dari penundaan buatan yang ditanamkan pada pvmbg-mock.

2.3. Komponen Sistem

Komponen	Tanggung jawab	Container	Port/API
BMKG mock	Menyediakan simulasi data seismik dan tsunami berlatensi rendah (50-150 ms) via REST API.	bmkg	8081
PVMBG mock	Menyediakan simulasi data vulkanik dengan latensi tinggi (500-3000 ms) dan mendukung simulasi kegagalan (*outage*).	pvmbg	8082
Aggregator	Bertugas memicu polling, memetakan skema ke *HazardEvent,* memegang kepemilikan akses basis data, dan mempublikasikan data (publish) ke message broker.	aggregator	Internal Network
Auth Service	Berperan sebagai pusat penerbitan, verifikasi *scope*, dan *refresh token* akses bagi klien *downstream*.	auth	Internal Network
Client-Facing API	Titik masuk tunggal (*gateway* publik) yang melayani permintaan baca dari klien Media, Tim Lapangan, dan Internal.	client-api	8080
Canonical Store	Basis data (fleksibel) penyimpan riwayat *HazardEvent* yang diakses secara mutlak hanya oleh Aggregator.	postgres	Internal (5432)
Message Broker	Infrastruktur pengirim pesan Pub/Sub yang menyalurkan setiap kejadian secara *fan-out* ke banyak *queue* tanpa *coupling* langsung.	rabbitmq	Internal (5672)
Dashboard consumer	Subscriber 1: Berlangganan data *HazardEvent* secara asinkron dari broker (mensimulasikan pembaruan dashboard BNPB).	dashboard-consumer	Internal (Worker)
Notifier consumer	*Subscriber* 2: Berlangganan data dari *broker* untuk mensimulasikan transmisi peringatan ke perangkat operasional.	notifier-consumer	Internal (Worker)

Diagram Arsitektur Sistem

3.1. Diagram



3.2 Narasi Arsitektur

Arsitektur sistem dibangun dengan pola *microservices* terdistribusi yang dijalankan di dalam kontainer terisolasi. Alur data dimulai dari komponen aggregator yang bertindak sebagai inisiator dengan melakukan *polling* secara berkala (2-5 detik) ke layanan bmkg-mockdan pvmbg-mock melalui protokol HTTP/REST. Setelah data mentah (*raw data*) diambil, aggregator memetakan dan menyatukan data tersebut ke dalam format kanonik HazardEvent.

Data kanonik tersebut kemudian disalurkan ke dua jalur:

- Persistensi Data: aggregator menyimpan data ke dalam Canonical Store. Basis data ini secara absolut hanya dimiliki dan dapat diakses langsung oleh aggregator. Layanan lain dilarang keras melakukan koneksi langsung ke pangkalan data ini.
- Distribusi Notifikasi: aggregator bertindak sebagai producer yang mempublikasikan (publish) event kedatangan data baru ke dalam Message Broker. Beberapa instansi layanan consumer berlangganan (subscribe) ke broker ini untuk menerima kejadian secara asinkron tanpa memberatkan tugas aggregator.

Di sisi akses pengguna luar, permintaan masuk melalui komponen client-api (berjalan di port 8080). Layanan client-api mendelegasikan proses verifikasi keabsahan dan cakupan (*scope*) dari *access token* klien (Media, Tim Lapangan, Internal Ops) ke komponen independen auth-service. Setelah diotorisasi, client-api mengambil data *HazardEvent* dengan cara memanggil internal API milik aggregator, menjaga prinsip isolasi penyimpanan. Di sepanjang siklus hidup permintaan lintas layanan ini, sistem menyertakan *correlation ID* unik pada *header* dan metrik log internal, serta mencatat beban latensi jaringan untuk keperluan observabilitas (*tracing*).

3.2.1. Seluruh Service dan Batas Container

Sistem terdiri dari kumpulan kontainer yang berdiri sendiri dan dapat di-*restart* tanpa saling menjatuhkan (*independent unit of deployment*):

- Source Mock Container: bmkg-mock (Port 8081) dan pvmbg-mock (Port 8082).
- Core Gateway Container: aggregator (komponen pusat pemrosesan), client-api (Port 8080 sebagai endpoint publik), dan auth-service (penerbit token).
- Event Driven Container: Komponen consumer (berjalan minimal 2 instansi independen).
- Infrastructure Container: Basis data penyimpanan (Canonical Store) dan sistem Message Broker.

3.2.2. Alur Event (Data Flow)

Aliran data bergerak dari hulu ke hilir dengan urutan kejadian sebagai berikut:

- Ingest: Container aggregator menembakkan permintaan polling secara berkala (interval 2-5 detik) ke bmkg-mock dan pvmbg-mock.
- Mapping: Data respons yang berwujud entitas Seismic Event dan Volcanic Report divalidasi dan ditransformasi (mapping) menjadi format standar HazardEvent di dalam memori aggregator. Atribut yang tidak terpetakan langsung dimasukkan ke dalam objek attributes.
- Persistence & Fan-out: Setelah data kanonik terbentuk, aggregator mengeksekusi dua tindakan: menyimpan rekam jejak tersebut ke Canonical Store, dan pada saat yang bersamaan memublikasikan (publish) data tersebut ke dalam sebuah topik di Message Broker.
- Consumption: Layanan consumer (seperti dashboard updater dan notifikasi lapangan) yang berlangganan pada broker tersebut akan menerima data secara asinkron (Pub/Sub) segera setelah event diterbitkan.

3.2.3. Kepemilikan Penyimpanan (Storage Ownership)

Menerapkan pola Database-per-Service, Canonical Store dimiliki dan dilindungi secara eksklusif oleh layanan aggregator. Container lain sama sekali tidak memiliki kredensial maupun akses jaringan langsung ke basis data ini. Jika komponen client-api ingin menyajikan data kepada pengguna (Media/Tim Lapangan), ia harus memanggil layanan REST internal milik aggregator, mematuhi aturan enkapsulasi data agar migrasi atau perubahan skema basis data di masa depan tidak merusak layanan lain.

3.2.4. Protokol Komunikasi Antar Service

- HTTP/REST API: Digunakan untuk komunikasi polling sinkron dari aggregator ke mock service (BMKG dan PVMBG). Protokol ini juga dipakai oleh klien eksternal untuk mengakses client-api, serta komunikasi internal antara client-api dengan auth-service dan aggregator.

- AMQP / Protokol Broker Asinkron: Digunakan pada alur distribusi HazardEvent terbaru dari publisher (aggregator) menuju message broker (misal: RabbitMQ), lalu diteruskan ke berbagai instansi consumer tanpa koneksi langsung (decoupled).

Daftar Teknologi Terpilih

Komponen / Kebutuhan	Teknologi	Alasan Pemilihan Terhadap Kebutuhan	Alternatif yang Dipertimbangkan & Trade-off
Service Implementation	**Golang** (Go)	Memiliki kapabilitas konkurensi bawaan (Goroutines dan Channels) yang sangat ringan. Sangat esensial untuk menyelesaikan Problem 2, di mana *polling* paralel diperlukan agar pemanggilan instansi BMKG yang cepat tidak terhambat (*head-of-line blocking*) oleh kelambatan PVMBG.	**Node.js**: *Single-threaded event loop* cukup baik untuk I/O asinkron, namun eksekusi *mapping* data masif (*CPU bound*) berpotensi menghambat antrean *event* jika dibandingkan dengan kompilasi *native* Golang.
Canonical Store	PostgreSQL (JSONB)	Penyimpanan ini mengakomodasi atribut dinamis tanpa pendefinisian struktur kaku. Hal ini mutlak diperlukan (Problem 4) agar saat instansi PVMBG tiba-tiba merilis kolom baru seperti confidence_level, sistem tidak perlu melakukan migrasi skema atau *downtime*.	**RDBMS Tradisional (Kolom Statis)**: Ditolak karena memaksa pelaksanaan DDL (*Data Definition Language*) seperti ALTER TABLE dan memicu keharusan *redeploy* secara bersamaan (melanggar prinsip *microservice* P4).
Message Broker	RabbitMQ	Menyediakan fitur antrean mandiri (*queue*) dan publikasi *fan-out* (Problem 5). Memungkinkan aggregator menyebarkan event secara asinkron tanpa perlu mengetahui atau menunggu respons dari ragam layanan consumer (*decoupled*).	**Direct HTTP/REST Call**: Ditolak karena menciptakan ikatan erat (*tight coupling*) antara *producer* dan *consumer*, di mana lambatnya satu klien penerima notifikasi akan menghambat *aggregator*.
Authentication	JSON Web Token (JWT)	Token bersifat nirstatus (*stateless*) yang dapat membungkus *scope* klien (seperti Media atau Tim	**Opaque Token dengan DB Sesi Terpusat (Redis)**:

		Lapangan) dan mengatur batas usia (*Time-To-Live*/TTL) token selama 60 detik (Problem 3). Hal ini mencegah paparan akses mentah secara tak berhak.	Ditolak karena menambah kompleksitas lapisan penyimpanan tambahan (Redis) hanya untuk memvalidasi setiap *request* yang masuk.
Containerization	Docker	Memenuhi mandat spesifikasi (Ketentuan Implementasi) di mana tiap entitas aplikasi wajib dikemas sebagai wadah mandiri yang memiliki Dockerfile tersendiri, memfasilitasi kebebasan mematikan dan *restart* satu layanan secara terpisah (Problem 4).	**Virtual Machine Lengkap (VM)**: Ditolak karena terlalu berat (*resource-intensive*) dan lambat dijalankan untuk sekadar layanan API modular.
Orchestration	Docker Compose	Memenuhi syarat implementasi yaitu menjalankan seluruh layanan terdistribusi secara paralel melalui satu injeksi perintah orkestrasi, yang terdefinisi pada berkas infra/docker-compose.yml.	**Kubernetes (K8s)**: Ditolak karena profil pembelajarannya dan *overhead* klastering (*Control Plane*) terlalu rumit serta berlebihan untuk ruang lingkup purwarupa Milestone 1.
Load Testing	k6 (JavaScript)	Berbasis *scripting* (scripts/load-test/sustained.js), memudahkan simulasi injeksi beban tingkat tinggi secara konstan (≥50 koneksi paralel) guna membuktikan p95 sistem tidak runtuh di bawah laju permintaan yang deras (Problem 2).	**Apache JMeter**: Ditolak karena konfigurasi berwujud XML yang kaku dan lebih lamban direvisi pada sistem *version control* dibandingkan dengan *script* JavaScript yang dinamis.

Asumsi

No.	Asumsi	Dampak terhadap desain/implementasi
1.	Keamanan Jaringan Internal Terisolasi Jaringan virtual internal yang dibentuk oleh Docker Compose bersifat aman dan terisolasi dari akses publik.	Komunikasi antar-service di back-end (misalnya dari client-api ke auth-service atau aggregator ke Message Broker) menggunakan HTTP/TCP biasa tanpa enkripsi Mutual TLS (mTLS). Pengamanan difokuskan pada otorisasi token di sisi client-facing dan header kredensial ke instansi sumber.
2.	Sinkronisasi Waktu Server (NTP) Seluruh kontainer di dalam sistem memiliki waktu yang tersinkronisasi (menggunakan zona waktu UTC standar).	Komponen auth-service dan client-api dapat memvalidasi kedaluwarsa Access Token (ber-TTL pendek 60 detik) secara akurat tanpa masalah perbedaan waktu antar-mesin. Demikian juga, stempel waktu ingested_at dan occurred_at dapat dibandingkan dengan presisi.
3.	Fokus Kegagalan Terpusat pada PVMBG Sesuai spesifikasi, layanan simulasi BMKG selalu stabil dan merespons dalam 50-150 ms, sedangkan skenario outage dan latensi ekstrem murni terjadi di PVMBG.	Logika degradasi terkendali (graceful degradation) dan strategi pembatasan waktu (timeout) pada aggregator dirancang secara spesifik untuk menangani anomali pada koneksi PVMBG. Jika PVMBG gagal, sistem otomatis melayani data rekaman vulkanik terakhir dengan indikator stale_since tanpa menahan respons data seismik BMKG.
4.	Kapasitas Penyimpanan dan Retensi Data Volume data (HazardEvent) yang dihasilkan selama evaluasi M1 tidak akan melampaui kapasitas penyimpan Canonical Store maupun Message Broker.	Sistem tidak mengimplementasikan mekanisme garbage collection, pembersihan data (data pruning), ataupun kebijakan retensi (penghapusan otomatis data lama). Seluruh riwayat rekaman pra dan pasca modifikasi skema confidence_level dibiarkan menumpuk di basis data.

Petunjuk Menjalankan Sistem

Ringkasan langkah-langkah untuk menjalankan dan menguji sistem tersedia di bawah ini. Panduan dan dokumentasi yang lebih lengkap merujuk pada berkas README.md di direktori utama repositori.

6.1. Prasyarat

Sebelum menjalankan sistem, pastikan lingkungan kerja telah memiliki perangkat lunak berikut:

- Go (opsional, jika ingin menjalankan service secara native di luar kontainer).
- Docker Desktop atau Docker Engine beserta plugin Docker Compose untuk orkestrasi kontainer.
- curl (atau Postman) untuk melakukan pemanggilan HTTP ke endpoint API.
- k6 terpasang di sistem untuk mengeksekusi skrip load testing.

6.2. Menyalakan Sistem

Sistem ini bergantung pada variabel lingkungan (*environment variables*) untuk manajemen kredensial dan konfigurasi. Salin *template* konfigurasi terlebih dahulu, kemudian jalankan Docker Compose:

```
# 1. Salin konfigurasi kredensial bawaan
cp .env.example .env

# 2. Bangun image dan jalankan seluruh container secara background
docker compose -f infra/docker-compose.yml up --build -d

# 3. Verifikasi bahwa seluruh layanan telah berstatus 'Up'
docker compose -f infra/docker-compose.yml ps
```

6.3. Health Check

Setiap layanan (*service*) menyediakan *endpoint* pemeriksaan kesehatan rutin (*health check*). Untuk memastikan instansi sumber (mock) telah berjalan dan siap menerima *polling* dari Aggregator, eksekusi perintah berikut:

```
curl -i http://localhost:8081/health
curl -i http://localhost:8082/health
```

6.4. Menjalankan Skenario Demonstrasi

Seluruh skenario pengujian didokumentasikan di README.md. Berikut adalah ringkasan perintah pemicu untuk menguji fungsionalitas utama (*Problem 1–5*):

- Pemicu Perubahan Skema (P1): Jalankan perintah curl -X POST http://localhost:8082/admin/schema-version untuk memunculkan kolom baru confidence_level pada data PVMBG.
- Pemicu Outage / Kegagalan (P2): Jalankan perintah curl -X POST http://localhost:8082/admin/outage untuk mematikan dan menyalakan kembali respons PVMBG secara instan.
- Pengujian Beban Concurrency (P2): Eksekusi skrip beban tinggi dengan menjalankan k6 run scripts/load-test/sustained.js.
- Pengujian Otentikasi (P3): Panggil endpoint client-api (Port 8080) dengan melampirkan JWT di header Authorization: Bearer <token>, dan tunggu selama >60 detik untuk melihat efek kedaluwarsa token bawaan.
- Pengujian Distribusi Pesan Pub/Sub (P5): Lakukan perintah docker compose -f infra/docker-compose.yml stop dashboard-consumer untuk mematikan satu pelanggan, lalu biarkan aggregator mengirim pesan. Jalankan kembali docker compose -f infra/docker-compose.yml start dashboard-consumer untuk melihat masuknya sisa *event* dari pesan RabbitMQ yang tertunda (*catch-up*).

Problem 1: Arsitektur & Interoperabilitas Data

7.1. Rancangan Solusi dan Mekanisme yang Dipilih

Untuk menangani perbedaan dan perubahan skema data antara BMKG dan PVMBG, komponen aggregator dirancang menggunakan metode *Dynamic Field Mapping* (Evolusi Skema Aditif). Pada kode Golang services/aggregator/main.go, JSON *response* dari *mock service* tidak di-*unmarshal* ke dalam struktur statis (*strict struct*) secara keseluruhan. Parameter utama kanonik (seperti hazard_id, source, occurred_at) dipetakan secara statis, sedangkan seluruh *field* yang tidak dikenali akan ditangkap ke dalam tipe data map[string]interface{} dan disimpan di bawah properti attributes pada objek HazardEvent.

7.2. Alasan Keputusan dan Trade-off

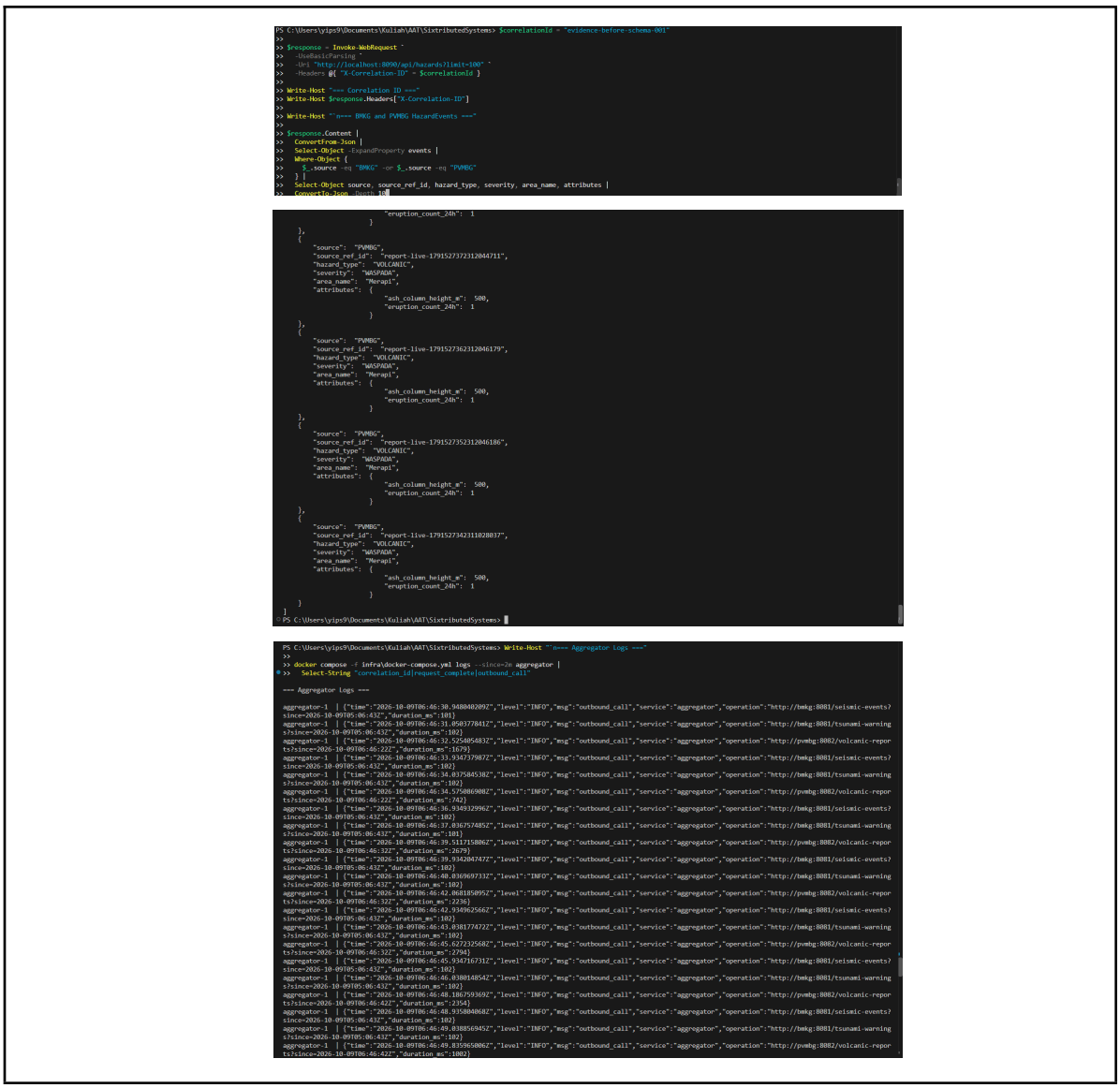
Alternatif yang dipertimbangkan: (1) Struktur statis (*Strict Struct*), dan (2) *Schema Registry* terpusat (seperti Apache Avro).

Keputusan dan Trade-off: Kami memilih *Dynamic Mapping* (JSON Map) karena memungkinkan sistem (aggregator) untuk beradaptasi dengan perubahan penambahan *field* (seperti confidence_level) secara otomatis tanpa mengalami *crash* atau membutuhkan *re-deploy* layanan. *Trade-off* dari keputusan ini adalah hilangnya validasi tipe data statis di dalam *field* attributes tersebut. Penggunaan *Schema Registry* dinilai terlalu kompleks (*overkill*) untuk ruang lingkup purwarupa basis HTTP JSON pada M1.

7.3. Batasan yang Disadari

Sistem ini secara sadar **hanya mendukung perubahan skema aditif** (penambahan atribut baru). Kami tidak menangani skenario destruktif seperti penghapusan atau pengubahan nama pada *field* identifikasi utama (contoh: report_id atau volcano_id). **Alasan:** Penghapusan parameter kanonik utama akan mencederai integritas relasional data di *Canonical Store* yang diwajibkan oleh kontrak desain.

7.4. Bukti Penyelesaian



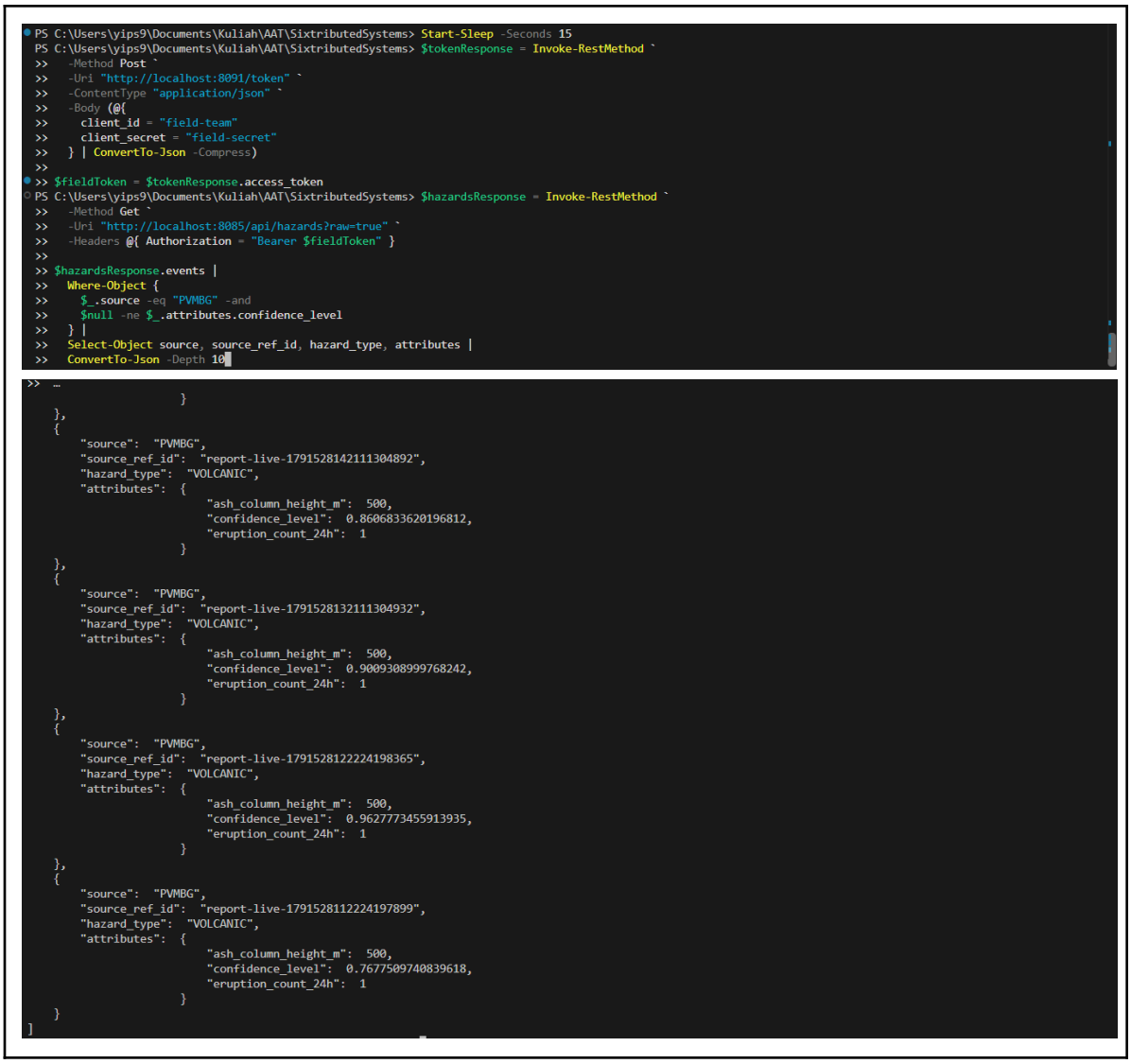
[Gambar 1.1] Tangkapan layar log terminal ber-*correlation ID* dari aggregator



```
PS C:\Users\yips9\Documents\Kuliah\AAT\SixtributedSystems> Invoke-RestMethod `
>>   -Method Post `
>>   -Uri "http://localhost:8082/admin/schema-version" `
>>   -Headers @{ Authorization = "Bearer example-pvmbg-token" } `
>>   -ContentType "application/json" `
>>   -Body (@{ enabled = $true } | ConvertTo-Json -Compress)
>>
>> docker compose -f infra\docker-compose.yml ps aggregator

NAME                IMAGE               COMMAND         SERVICE      CREATED          STATUS          PORTS
infra-aggregator-1  infra-aggregator    "/aggregator"   aggregator   45 minutes ago   Up 45 minutes   0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp
schema_version
--------------
          True
```

[Gambar 1.2] Tangkapan layar pemanggilan curl -X POST /admin/schema-version



[Gambar 1.3] Tangkapan layar respons API JSON (dari *client-api*)

7.5. Jawaban atas Butir Di Laporan

Mekanisme yang Dipilih: *Dynamic JSON mapping* menggunakan tipe Map di Golang.

Kebijakan Field Tak Dikenal: Seluruh *field* baru yang tidak ada pada skema dasar kanonik secara otomatis diteruskan dan disematkan ke dalam objek `attributes`.

Dukungan Perubahan: Mendukung penambahan *field* baru (*additive*). Tidak mendukung penghapusan *field* identitas utama dan inkompatibilitas perubahan tipe data dasar.

Problem 2: Concurrency, Availability, & Graceful Degradation

8.1. Rancangan Solusi dan Mekanisme yang Dipilih

Untuk menghentikan *head-of-line blocking* akibat latensi tinggi PVMBG (hingga 3 detik), aggregator mengeksekusi penarikan data (*polling*) secara asinkron menggunakan instrumen konkurensi Golang, yaitu *Goroutines* dan *WaitGroups*. Panggilan ke BMKG dan PVMBG dilakukan secara paralel. Jika layanan PVMBG mengalami gangguan (*outage*), aggregator akan menangkap *timeout* HTTP, lalu melakukan mekanisme *fallback* ke basis data untuk mengambil status erupsi terakhir yang disisipkan atribut tambahan stale_since.

8.2. Alasan Keputusan dan Trade-off

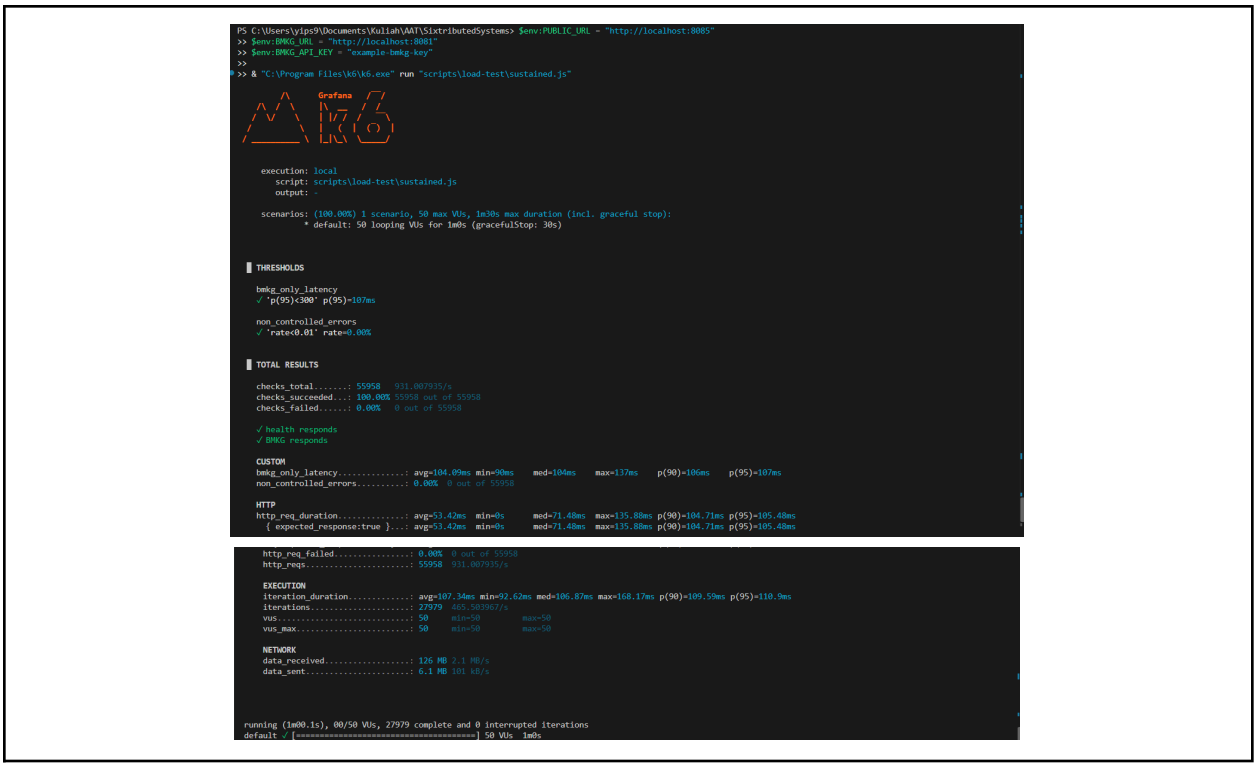
Alternatif yang dipertimbangkan: *Sequential Polling* (sinkron) dan pemrosesan berbasis Antrean (*Queue-based processing*).

Keputusan dan Trade-off: *Sequential polling* kami tolak karena lonjakan latensi PVMBG akan memaksa pembaruan data BMKG ikut tertunda. Kami memilih *Goroutines* paralel dengan batasan konkurensi (*Bounded Concurrency* / *Rate Limiting* 429) untuk melindungi sistem kehabisan memori (*resource exhaustion*) ketika volume *request* membludak.

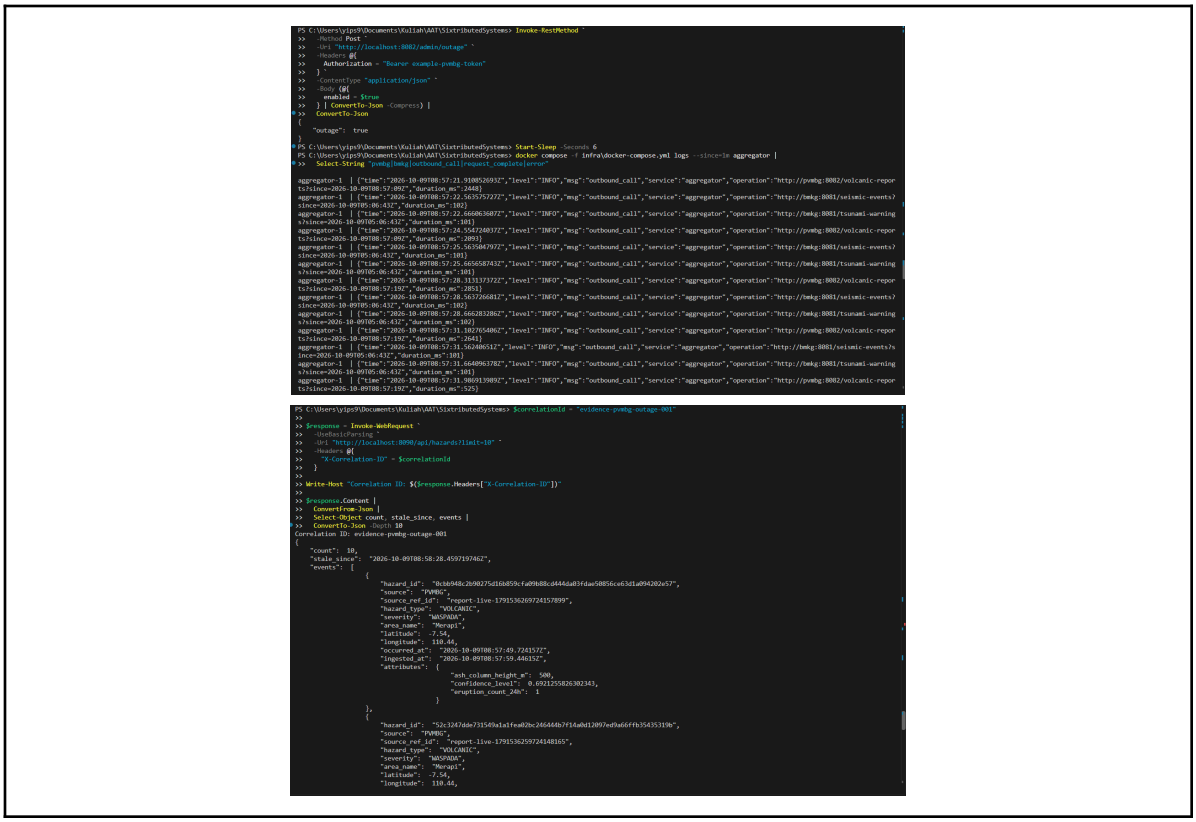
8.3. Batasan yang Disadari

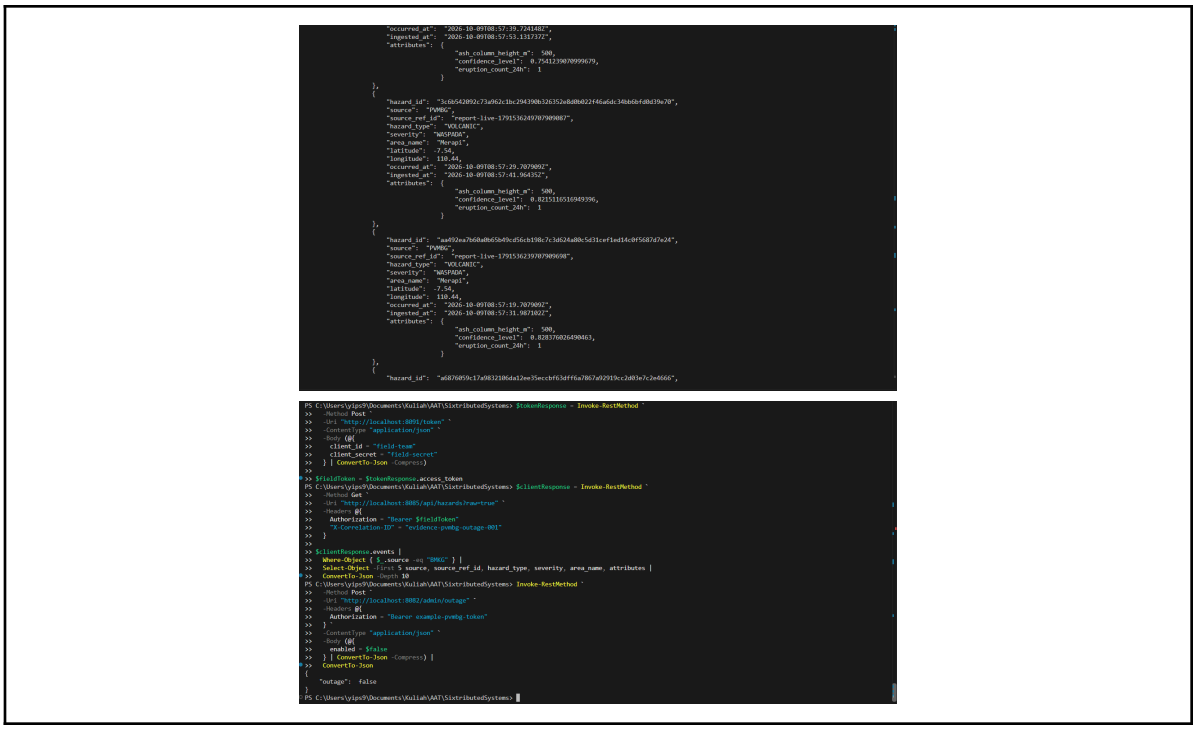
Mekanisme degradasi sistem (*graceful degradation*) ini tidak mengimplementasikan *Circuit Breaker* tingkat lanjut (seperti perpustakaan *Hystrix*), melainkan sekadar mengandalkan *timeout* batas HTTP dan *fallback* rekaman terakhir di basis data. **Alasan:** Mempertahankan dependensi *service* seminimal mungkin demi mematuhi batas arsitektur dasar Milestone 1.

8.4. Bukti Penyelesaian



[Gambar 2.1] Tangkapan layar keluaran metrik dari utilitas k6 (scripts/load-test/sustained.js)





[Gambar 2.2] Potongan log aggregator dan client response ketika skenario POST /admin/outage dipicu pada PVMBG

8.5. Jawaban atas Butir Di Laporan

Model Konkurensi: Menggunakan *Parallel Polling* asinkron melalui eksekusi *Goroutines* pada Golang.

Konfigurasi Load Testing: Menggunakan utilitas k6 dengan 50 pengguna virtual (*Virtual Users* / koneksi paralel) selama durasi 60 detik (*sustained*) yang menyerang *endpoint* campuran.

Problem 3: Service-to-Service Authentication & Trust

9.1. Rancangan Solusi dan Mekanisme yang Dipilih

Domain keamanan dipisahkan menjadi dua lapisan yang ditangani oleh auth-service dan aggregator.

Keamanan Internal ke Source: Kredensial untuk BMKG (X-BMKG-Key) dan PVMBG (Authorization: Bearer) diinisialisasi melalui *Environment Variables* di sisi aggregator tanpa terekspos ke klien eksternal.

Keamanan Eksternal (Client): Pengguna akhir mengotentikasi diri ke auth-service untuk mendapatkan *JSON Web Token* (JWT). JWT ini memiliki masa berlaku yang sangat singkat (TTL 60 detik) dan memuat *klaim scope* spesifik (Media, Tim Lapangan, Internal Ops) untuk membatasi cakupan *field* (ringkasan vs raw).

9.2. Alasan Keputusan dan Trade-off

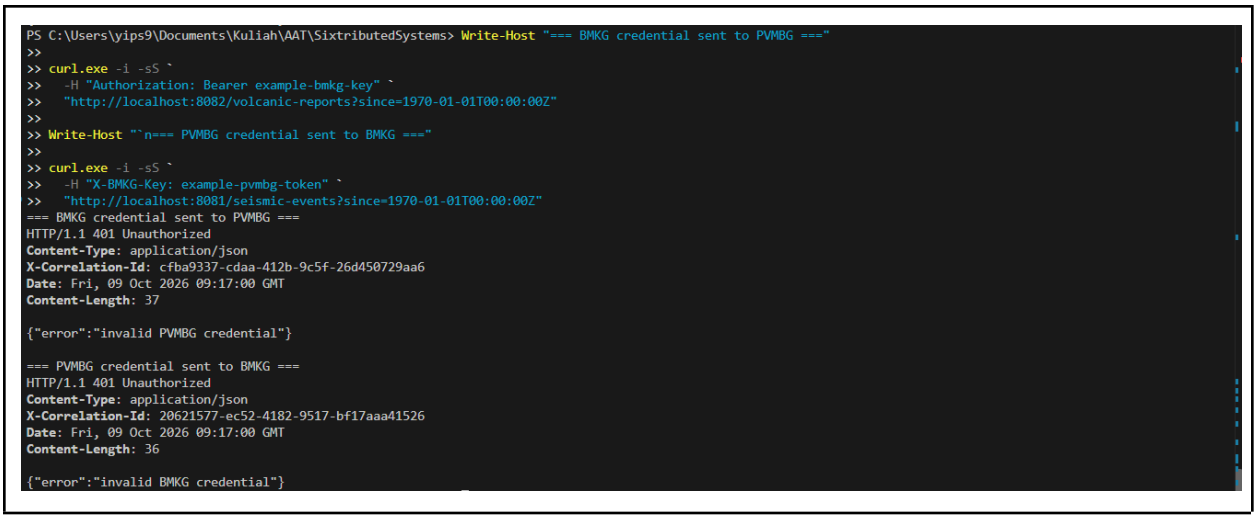
Alternatif yang dipertimbangkan: *Opaque token* tersentralisasi berbasis sesi *Redis* dan penerapan peran global secara absolut.

Keputusan dan Trade-off: Kami memilih menggunakan JWT nirstatus (*stateless*) untuk mengurangi hambatan interkomunikasi pemeriksaan *database* pada setiap pemanggilan client-api. *Trade-off* dari ketiadaan mekanisme *Redis* ini diakali dengan menetapkan jendela masa berlaku (*TTL*) token sekecil mungkin (60 detik) agar jendela eksploitasi token yang dicuri dapat diminimalisir.

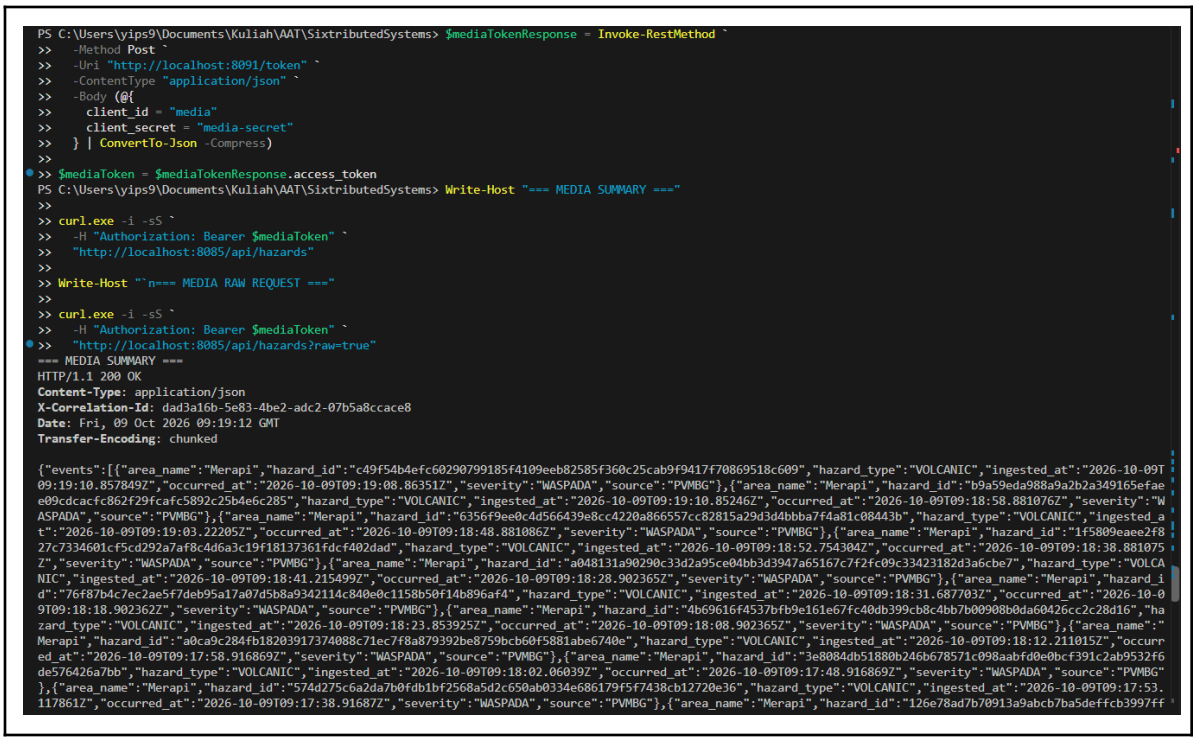
9.3. Batasan yang Disadari

Tidak tersedianya implementasi daftar pencabutan token (Token *Revocation List* / *Blacklist*). **Alasan:** Jika suatu akun Media diretas, akses baru akan tertutup ketika token 60 detiknya habis. Implementasi *blacklist* memori pada *node* akan bertentangan dengan pendekatan *stateless* pada desain dasar M1.

9.4. Bukti Penyelesaian

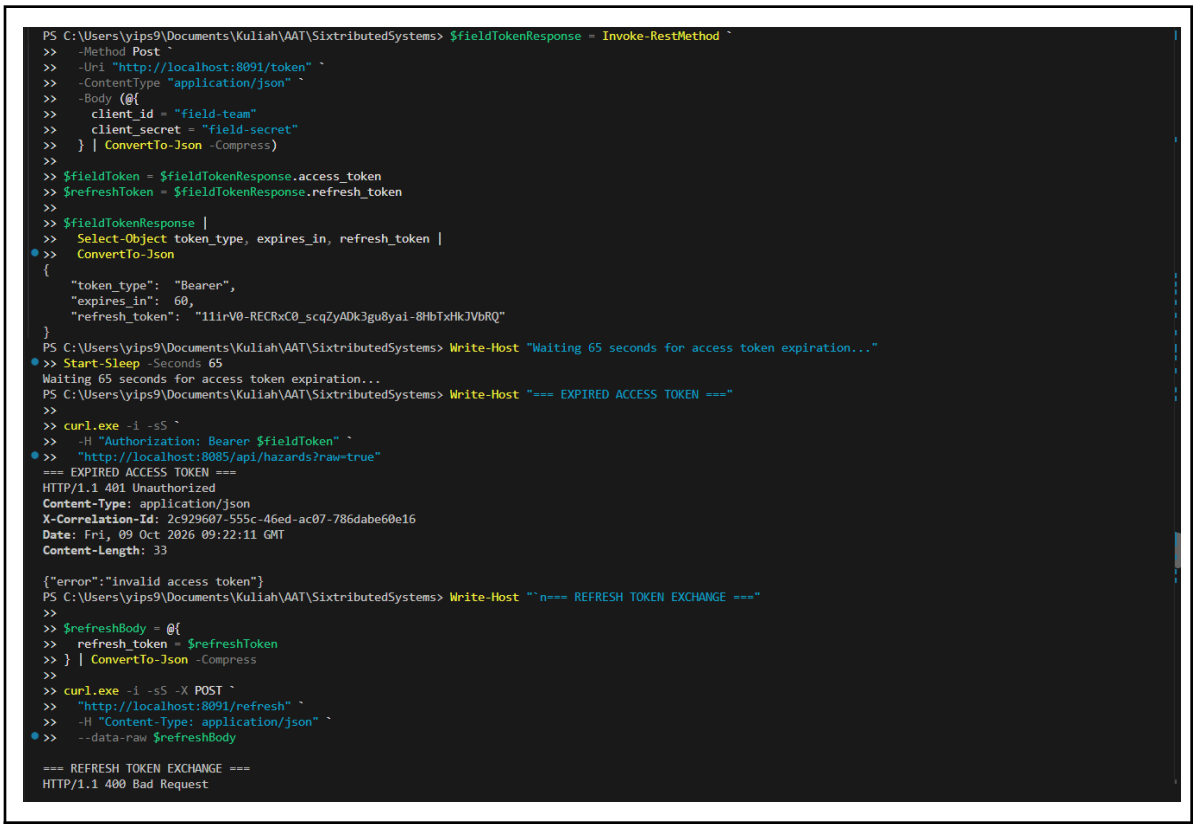


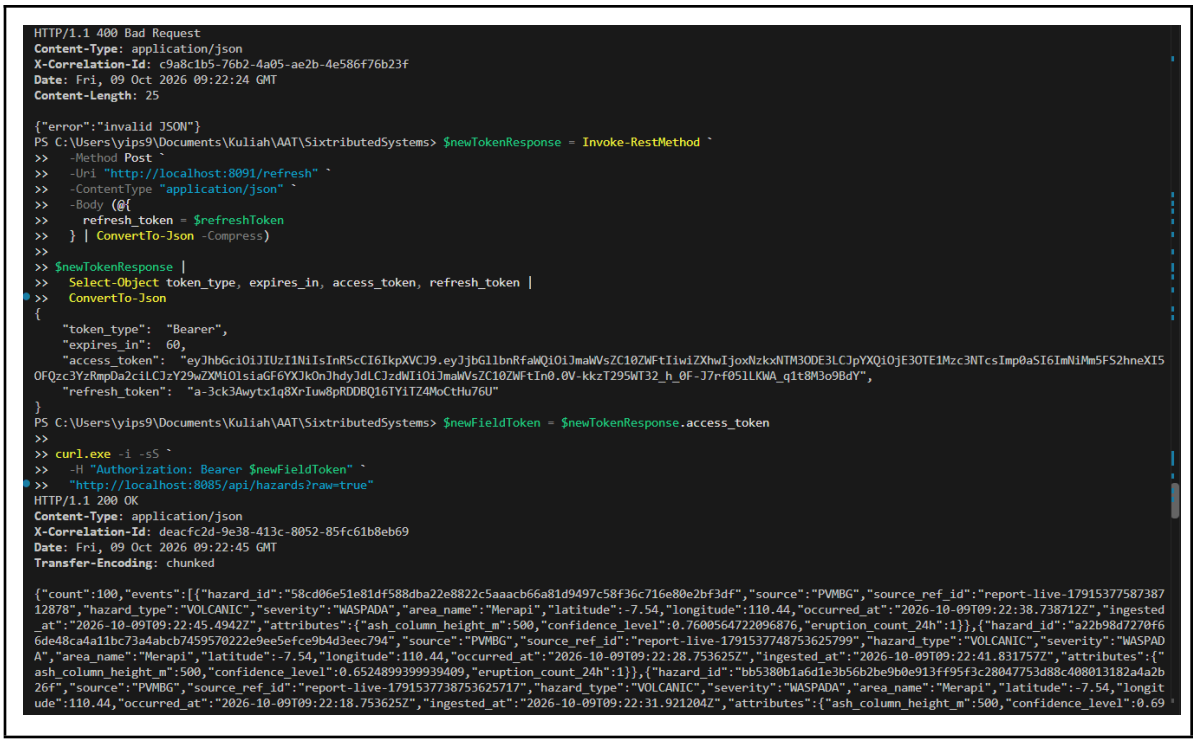
[Gambar 3.1] Respons API (HTTP 401/403) yang membuktikan kredensial konfigurasi token BMKG ditolak oleh *mock service* PVMBG.





[Gambar 3.2] Respons API client-api dengan JWT *role* Media yang mendapatkan *summary* dan tertolak (HTTP 403) ketika memaksa meminta *raw data*.





[Gambar 3.3] Potongan Log / Tangkapan Layar Postman yang menampilkan proses pertukaran *Refresh Token* yang berhasil sesudah 60 detik.



[Gambar 3.4] Pembuktian bahwa *repository* tidak menyimpan berkas rahasia, melainkan hanya menyediakan *template* .env.example.

9.5. Jawaban atas Butir Di Laporan

Mekanisme yang digunakan adalah *Domain of Trust Isolation* antara kredensial instansi (diatur murni dari *backend*) dan klien (*role-based* JWT). Keputusan mendelegasikan validasi melalui JWT ber-TTL pendek ini tidak melahirkan celah keamanan baru, sebab otorisasi verifikasi klaim dipusatkan di *Gateway API*, memastikan tidak ada jalan pintas ke data spesifik tanpa tanda tangan valid dari auth-service.

Problem 4: Arsitektur Microservice & Flexible Storage

10.1. Rancangan Solusi dan Mekanisme yang Dipilih

Sistem mematuhi paradigma *Single Responsibility Principle* dengan membagi sistem menjadi kumpulan *microservices*: aggregator, client-api, auth, dashboard-consumer, dan notifier-consumer. Orkestrasi antar-lingkungan ditangani oleh berkas infra/docker-compose.yml. Khusus penyimpanan *Canonical Store*, kepemilikan data (R/W) diisolasi mutlak hanya di *service* aggregator. Skema penyimpanan *database* menggunakan basis relasional **PostgreSQL** dengan pemanfaatan tipe data **JSONB** untuk mengakomodasi fleksibilitas atribut dinamis secara bawaan (menyimpan *field* attributes).

10.2. Alasan Keputusan dan Trade-off

Alternatif yang dipertimbangkan (Batas Service): Sistem *monolithic*.

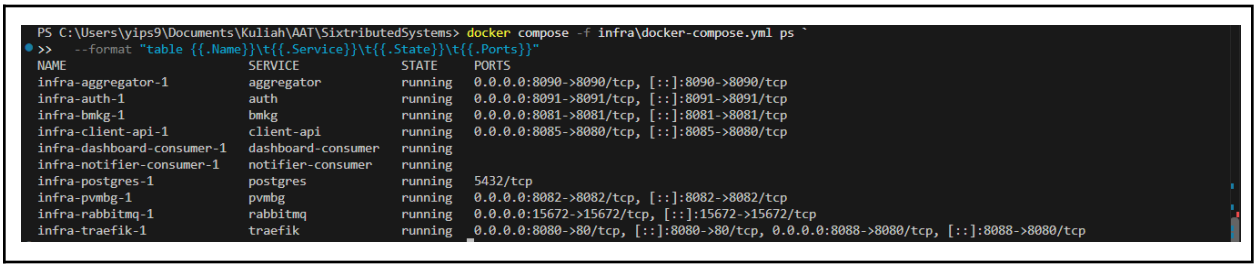
Alternatif yang dipertimbangkan (Storage): Basis data relasional (SQL) berbasis kolom statis.

Keputusan dan Trade-off: Pemisahan *services* meminimalisir *blast radius*; kerusakan pada auth-service tidak akan mematikan proses *polling* di aggregator. Dari segi penyimpanan, opsi RDBMS kolom statis ditolak sebab kemunculan *field* mendadak seperti confidence_level akan memaksa migrasi *ALTER TABLE* manual (*downtime*). Oleh karena itu, basis data berbasis dokumen/JSON dipilih sebagai penampung struktur bebas attributes tanpa mengorbankan *uptime*.

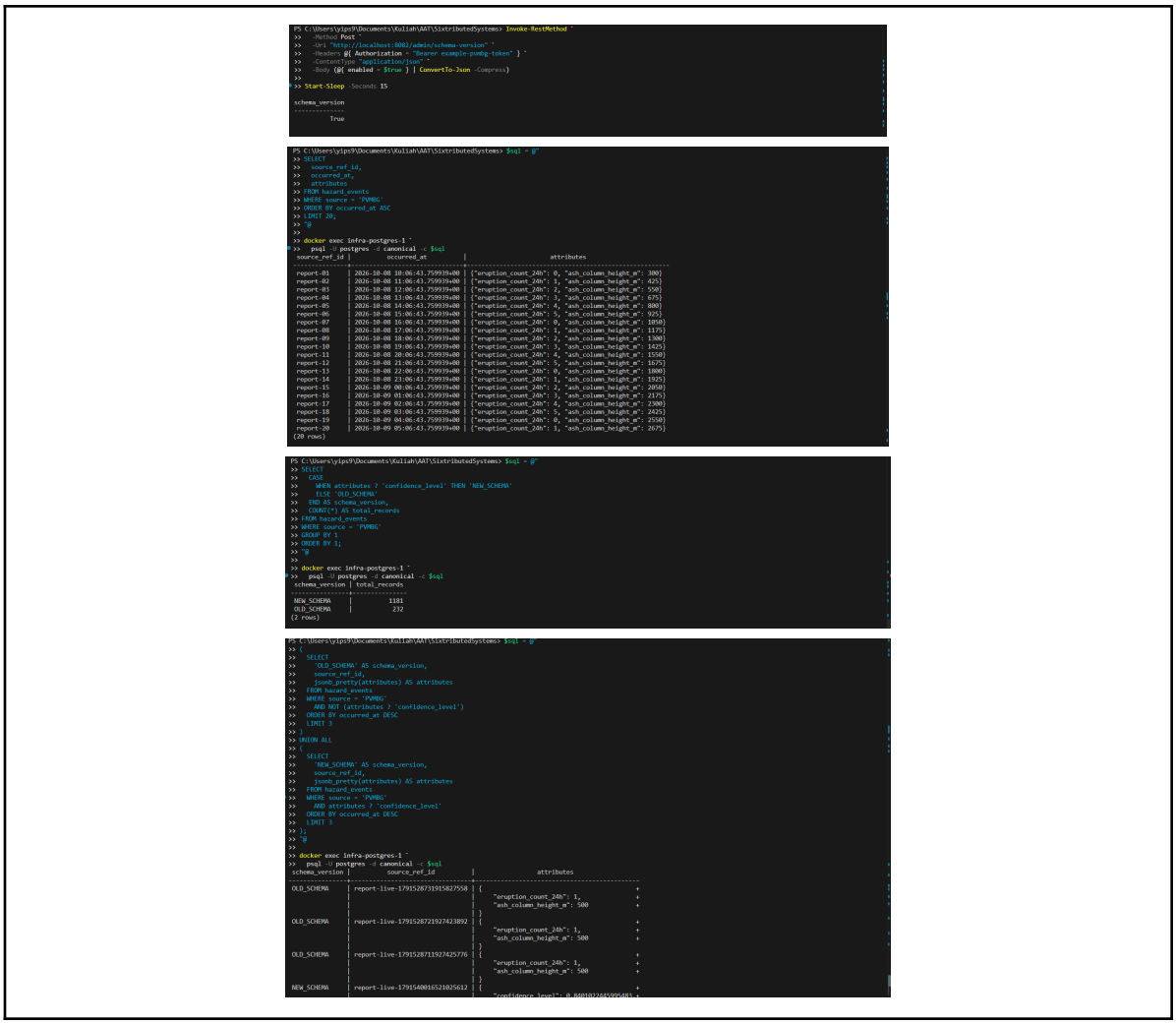
10.3. Batasan yang Disadari

Kurangnya dukungan tipe data yang ketat (*type safety*) pada kolom ekstensi dinamis attributes pada pangkalan data. **Alasan:** Sifat alami format JSON/NoSQL yang fleksibel membuat validasi konsistensi struktur terpaksa dialihkan bebannya ke lapisan kode aplikasi (Golang), alih-alih mesin *database*.

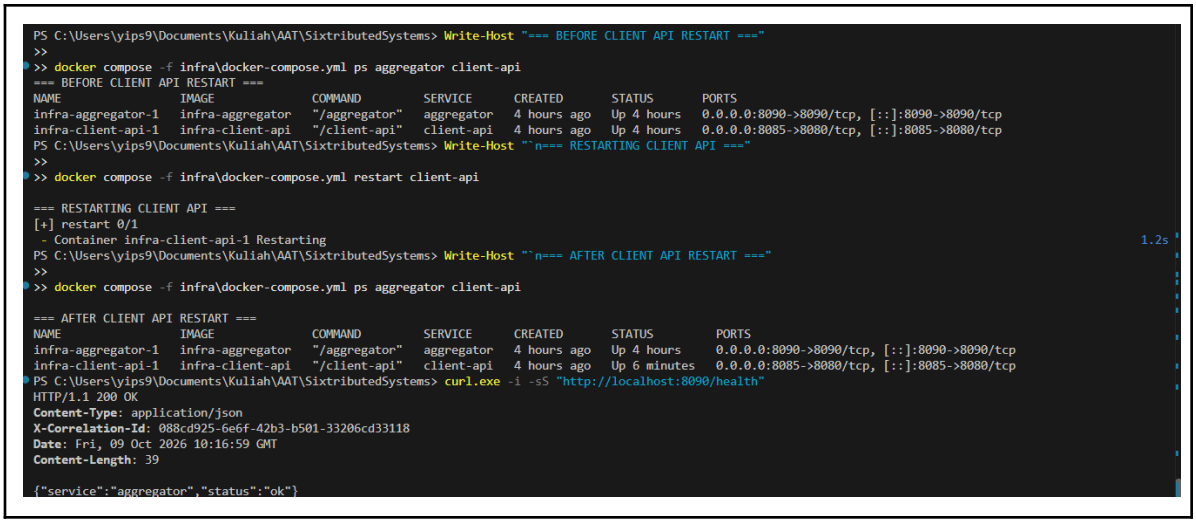
10.4. Bukti Penyelesaian

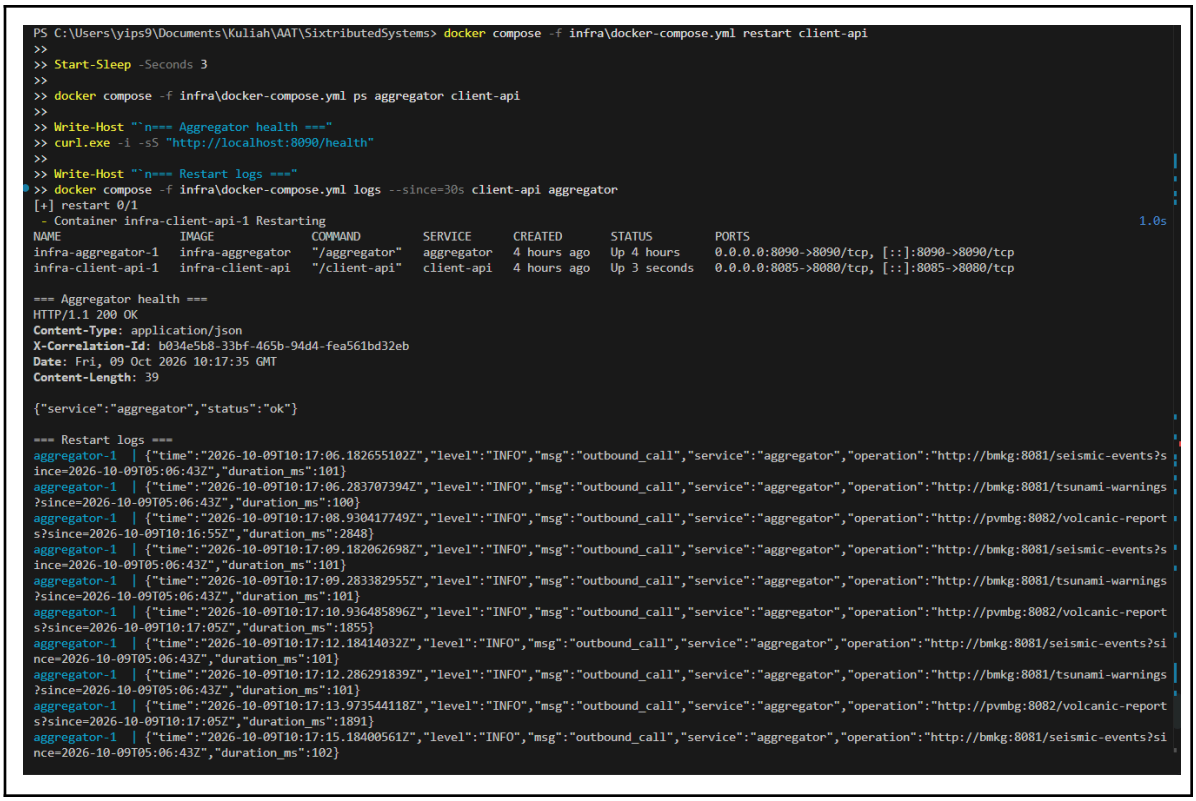


[Gambar 4.1] Tangkapan layar keluaran docker compose ps yang mengindikasikan setiap layanan berjalan di penampung (*container*) yang berbeda.



[Gambar 4.2] Tangkapan layar dari antarmuka/klien basis data yang menunjukkan dokumen/data dengan format lama berdampingan secara aman dengan data versi skema baru.





[Gambar 4.3] Potongan log di mana satu layanan (misal: client-api) mengalami inisialisasi/ulang *restart*, dan layanan aggregator tetap hidup tanpa gangguan.

10.5. Jawaban atas Butir Di Laporan

Alternatif Penyimpanan: RDBMS Statis vs Penyimpanan Dokumen (JSONB/NoSQL).

Alasan Keputusan Batas Service: Memastikan tidak ada *coupling* memori; penyesuaian fungsional otentikasi tidak mewajibkan proses *redeploy* komponen vital *polling*. *Canonical Store* menggunakan basis data berjenis dokumen untuk memungkinkan mutasi *field* secara aditif tanpa *downtime* skrip migrasi.

Problem 5: Fan-out Event Distribution / Pub-sub

11.1. Rancangan Solusi dan Mekanisme yang Dipilih

Distribusi *event* ke sistem *downstream* memanfaatkan pola *Pub/Sub*. aggregator (sebagai *publisher*) mengirimkan muatan data dari *HazardEvent* menuju *Exchange* di dalam piranti *Message Broker* **RabbitMQ**. Terdapat dua *instance service consumer*, yaitu **dashboard-consumer** dan **notifier-consumer**, yang secara independen memiliki *queue* masing-masing dan berlangganan (*subscribe*) pada saluran RabbitMQ tersebut untuk keperluan pembaruan dasbor visual dan penyebaran notifikasi peringatan.

11.2. Alasan Keputusan dan Trade-off

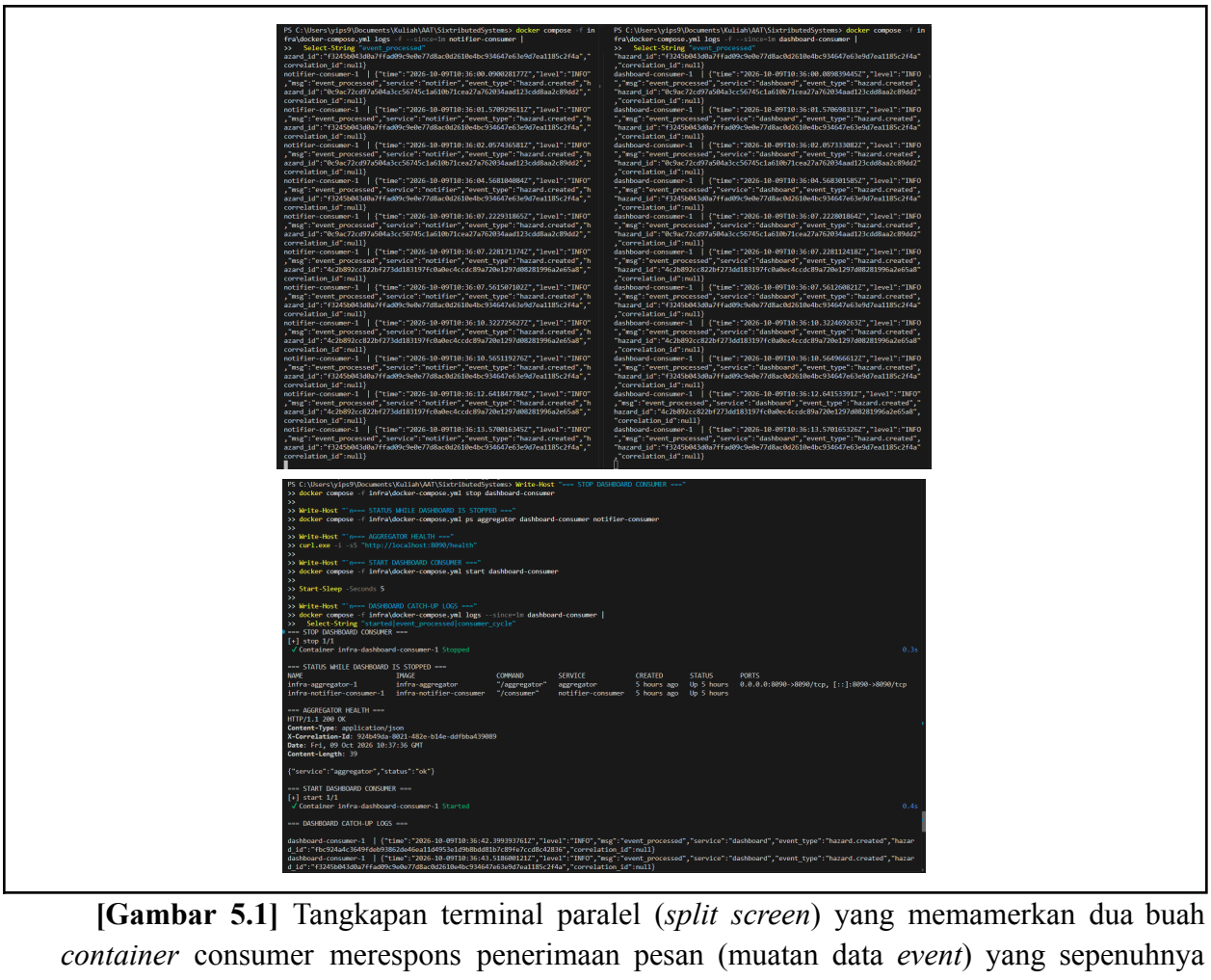
Alternatif yang dipertimbangkan: Integrasi sinkron melalui *HTTP Webhook Direct Call* ke semua klien.

Keputusan dan Trade-off: Panggilan Webhook sinkron ditolak secara tegas lantaran hal ini akan menahan (*block*) siklus kerja *Aggregator* ketika salah satu konsumen melambat atau sedang mati. Penggunaan perantara antrean *Broker* (*RabbitMQ/Kafka*) bersifat *fire-and-forget* dari sisi produsen. *Trade-off* mekanisme ini adalah peningkatan kompleksitas topologi jaringan pada orkestrasi docker-compose.yml akibat penambahan *container broker* eksternal.

11.3. Batasan yang Disadari

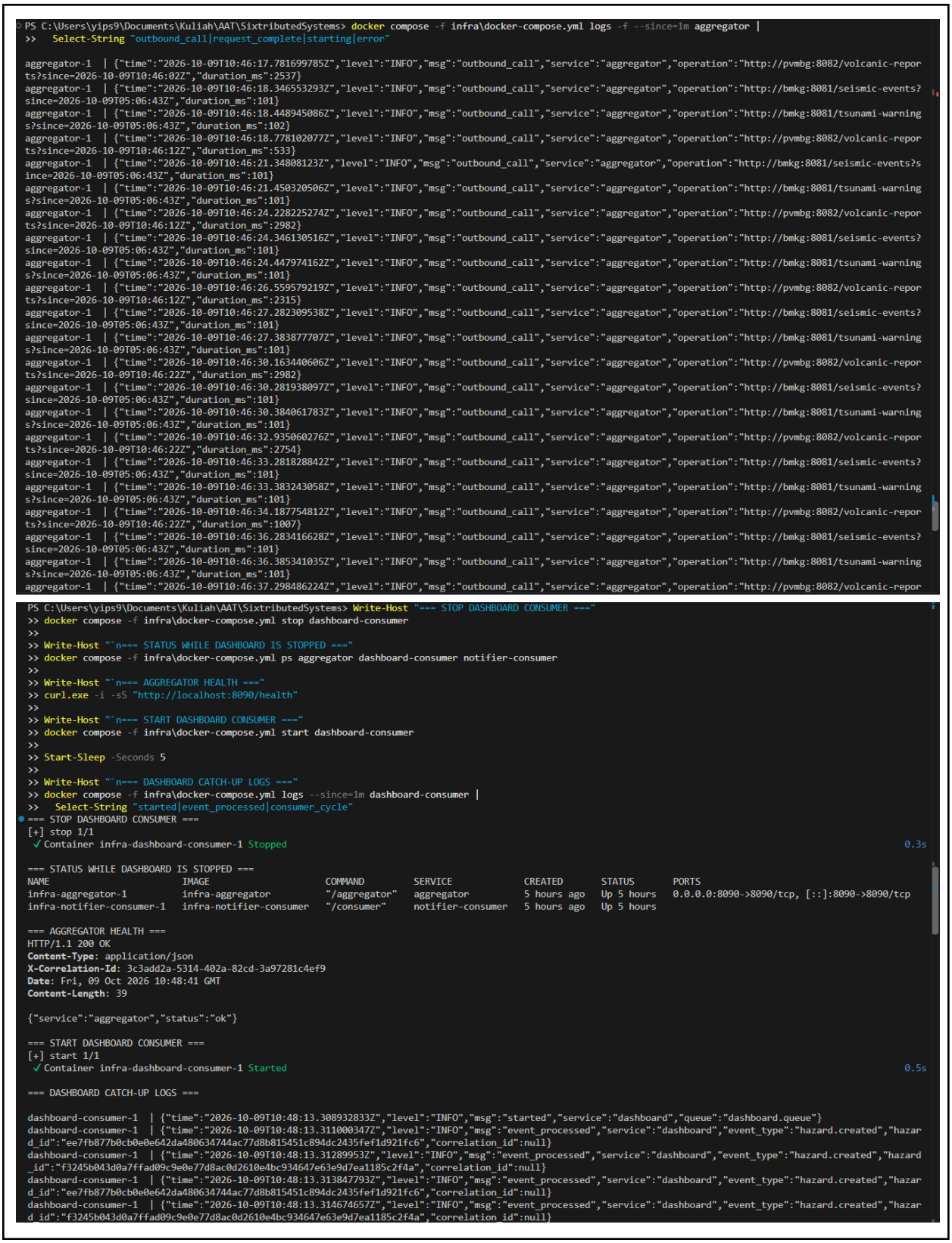
Mekanisme jaminan pengiriman hanya terjamin di taraf minimum (contoh: *At-Least-Once*). **Alasan:** Upaya meniadakan pesan duplikat (*Exactly-once delivery*) membutuhkan perancangan sistem deduplikasi *(idempotency)* memori yang teramat kompleks pada *consumer*, yang belum diimplementasikan dengan sempurna pada lingkup Milestone 1.

11.4. Bukti Penyelesaian



[Gambar 5.1] Tangkapan terminal paralel (*split screen*) yang memamerkan dua buah *container* consumer merespons penerimaan pesan (muatan data *event*) yang sepenuhnya sama di detik yang bersamaan.





[Gambar 5.2] *Log trace* ketika salah satu consumer secara paksa dihentikan (docker stop), aggregator membuktikan kemampuannya untuk terus menerbitkan *event* baru tanpa

macet, dan konsumen pengganti menangkap tumpukan *event* tertunda *(catch-up)* saat dihidupkan kembali.

11.5. Jawaban atas Butir Di Laporan

Broker yang dipilih: RabbitMQ

Semantik Pengiriman: *At-least-once*

Konsekuensi & Idempotensi: Karena semantik *at-least-once* membuka peluang satu pesan diteruskan lebih dari sekali *(duplicate delivery)* jika sambungan *acknowledge* terputus, komponen consumer selayaknya dirancang bersifat idempoten (contoh: melakukan pengecekan rekaman hazard_id sebelum mengirim surel) untuk mencegah redudansi tindakan.

DAFTAR PUSTAKA